

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №1

Выполнила:

Красноперова Елизавета

К3343

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Используйте любой boilerplate, который считаете наиболее подходящим для ваших нужд.

С учётом имеющихся данных по результатам ДЗ1 и ДЗ2 реализуйте:

- модели
- представления
- контроллеры

В результате выполнения лабораторной работы вы должны продемонстрировать полностью работающее API согласно вашего варианта.

Ход работы

На основе спроектированной в ДЗ1 базы данных и разработанной в ДЗ2 OpenAPI-схемы был выбран стек технологий: Node.js + Express + TypeORM + PostgreSQL. TypeORM позволяет описать модели и связи декларативно через декораторы, что идеально совпадает с уже спроектированной ER-диаграммой. Express обеспечивает простую и быструю маршрутизацию REST-эндпоинтов без лишнего кода. PostgreSQL - надёжная реляционная СУБД, которая поддерживает enum-типы и сложные запросы с JOIN, необходимые для фильтрации рецептов по ингредиентам.

Была создана структура проекта:

src/entity/ - модели (сущности TypeORM);

src/controller/ - контроллеры с бизнес-логикой;

src/routes/ - маршруты API;

src/middleware/ - промежуточное ПО (JWT-авторизация);

src/index.ts - точка входа, подключение к БД.

Модели, реализованные в соответствии с ERD:

User – пользователь;

Recipe – рецепт;

Step - шаг приготовления;

Ingredient - ингредиент (справочник);

RecipeIngredient - состав рецепта;

Comment - комментарий);

Like – лайк;

SavedRecipe - сохранённый рецепт;

Subscription - подписка на автора.

Контроллеры, реализованные согласно эндпоинтам из ДЗ2:

AuthController:

- POST /api/auth/register - регистрация с хэшированием пароля (bcrypt) и выдачей JWT-токена
- POST /api/auth/login - вход с проверкой пароля и выдачей JWT-токена

RecipeController:

- GET /api/recipes - поиск с фильтрацией по типу блюда, сложности, времени приготовления, ингредиентам, сортировкой и пагинацией
- GET /api/recipes/:id - страница рецепта с шагами, ингредиентами, автором, количеством лайков и комментариев, флагами is_liked_by_me и is_saved_by_me
- POST /api/recipes - создание рецепта (только авторизованные)
- PUT /api/recipes/:id - редактирование (только автор)
- DELETE /api/recipes/:id - удаление (только автор)

UserController:

- GET /api/users/me - профиль текущего пользователя
- PUT /api/users/me - редактирование профиля
- GET /api/users/me/recipes - мои опубликованные рецепты
- GET /api/users/me/saved - сохранённые рецепты

SocialController:

- POST/DELETE /api/recipes/:id/like - лайк/снятие лайка
- POST/DELETE /api/recipes/:id/save - сохранение/удаление из закладок
- GET/POST /api/recipes/:id/comments - список и создание комментариев
- PUT/DELETE /api/comments/:id - редактирование и удаление комментариев (только автор)
- POST/DELETE /api/users/:id/subscribe - подписка/отписка
- GET /api/users/me/subscriptions - мои подписки
- GET /api/users/me/subscribers - мои подписчики

Реализован authMiddleware для проверки JWT-токена в заголовке Authorization: Bearer <token>. Все защищённые эндпоинты помечены (security: BearerAuth: []). Настроено подключение к PostgreSQL через TypeORM с автоматической синхронизацией схемы БД (synchronize: true). При запуске сервера все таблицы создаются автоматически. Имена полей в сущностях совпадают с именами колонок в ER-диаграмме.

Вывод

В результате выполнения лабораторной работы реализовано полностью функционирующее REST API для сервиса обмена рецептами и кулинарных блогов. Сервер запускается без ошибок, подключается к PostgreSQL, автоматически создаёт таблицы. Все заявленные в задании функции работают.